

The Trireme

The trireme was the fastest warship of the ancient world, not because of its size, but because of its structure. Three banks of oarsmen, each pulling in coordination. Remove one bank and it slows. Remove two and it barely moves. The power was never in the number. It was in the arrangement.

If predictability is the proof, what kind of team produces it?

So what does the team actually look like?

Team sizes have always been a function of the bottleneck. In traditional agile, 7 to 10 people was the recommended unit, calibrated to sustain an implementation pipeline. The number was the answer to a specific question: how many people does it take to keep implementation moving?

Agents change the answer, because they change the question. The Spec Session is now the bottleneck. It gets slower as the session fills up and convergence takes longer to build a shared picture.

So the logic that produced 7-to-10 person teams no longer applies. The right team size is now the one that can run an effective Spec Session.

The Spec Session needs three kinds of attention in the room.

Someone who holds the system direction, the direction work from the End of a Craft chapter, now named as a role: the person who tracks what the system is becoming and whether the decisions being made are consistent with that, regardless of title or seniority. In a small team this is often the same person who runs the Spec Session. But there needs to be someone in every team.

Someone who watches the ground. The behavioral edge cases, the implementation details, the

things that drift when nobody is looking. This is often where juniors do their most valuable work. The junior catching what the senior's attention has moved past, the dynamic from the Fool with a Tool chapter, is by design.

Someone who manages the outside. Stakeholder pressure, organizational noise, the requests that arrive before anyone has had time to spec them. The Stakeholder Navigator from the Product Owner is Dead chapter. With that role filled, the room stays protected long enough to think.

The agent is the fourth, running only after the other three have done their work.

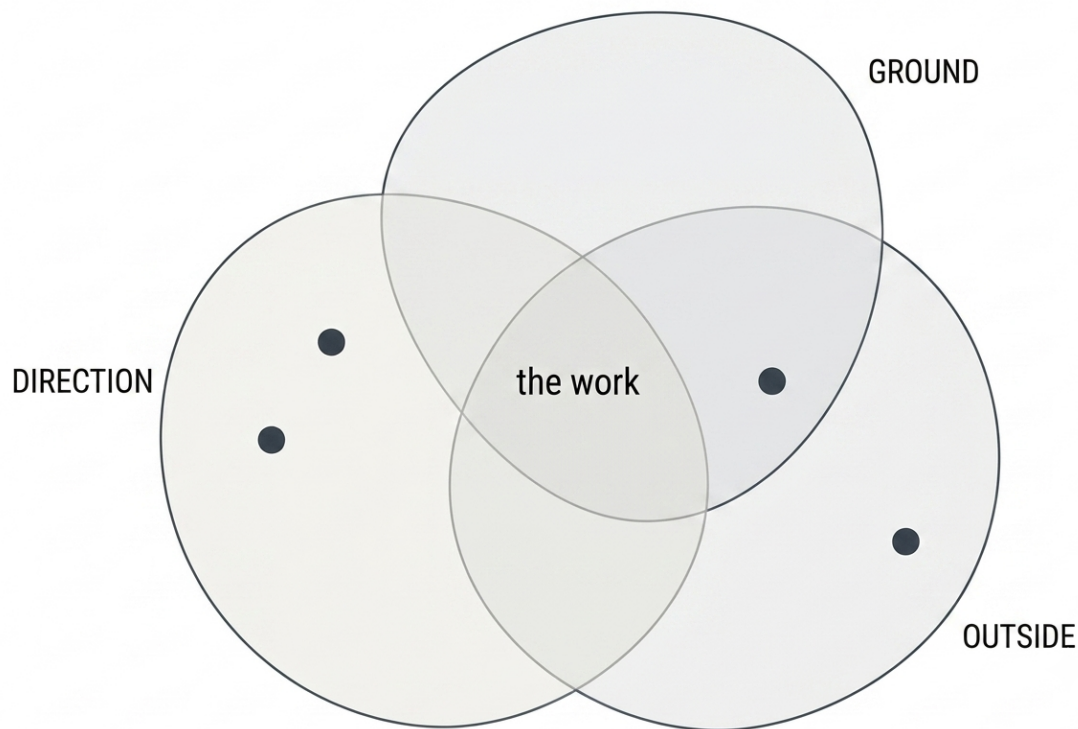


Figure 1: Direction, ground, and outside overlap into the work; the agent runs only in that overlap.

I've watched these functions show up without anyone assigning them.

In OpenFeature, a contributor once noticed something the maintainers had stopped seeing. Certain providers broke in a subtle way when reused, not because the code was wrong, but because it relied on something the code never said. If you knew the assumption, everything worked. The code didn't carry the assumption. The people did.

The cheap fix was obvious. Write it down. A documentation change, a warning in the README.

We were close to taking it. Then one of the maintainers pushed back. An implicit contract that lives in documentation is a contract that fails the moment someone builds without reading it, and someone always builds without reading it. He argued for an explicit interface instead. More

work now, less archaeology forever.

None of this happens unless someone accepts the issue first, looking at a report from outside the project and deciding it deserves the project's attention. That sounds like housekeeping. It isn't. It's the seat where the project meets everyone who uses it and will never show up in person.

Three kinds of attention, and nobody assigned any of them. Someone with eyes on the ground, catching what familiarity had made invisible. Someone answering to the people outside the room. Someone holding direction, thinking past this pull request into the years after it. Not roles in a process document. The structure of the project just makes them show up.

And underneath them, redundancy. The fix didn't go through on one person's say-so. Other maintainers weighed in, pushed on the interface argument, approved it. No single mental model decided the outcome, and no single departure could have taken the understanding with it.

So how many people does that take? As many as the process requires, as few as it can sustain. The functions don't answer that: the seats overlap, and one person can hold two. Two people can build a shared understanding, but a pair converges: the angles that started different grow shared, until nobody is left outside the framing. Three creates the triangle, the space where a third perspective surfaces what two people, heads down together, stop seeing.

One person holding critical system knowledge is a single point of failure. Two creates a dependency. Three creates a floor that survives someone going heads down, someone leaving, someone getting pulled onto something else.

The minimum viable unit is three.

Still, some organizations will operate at 15 or 20 people, for good reasons: scale, domain breadth, regulatory complexity. What changes is how those teams structure themselves internally.

The model that works is stream-aligned teams¹: smaller units, each running their own Spec Session on their slice of the problem, each holding deep shared understanding within their domain. Subject matter experts who own their territory. The larger team doesn't need a single room where everyone understands everything. It needs enough boundary knowledge between units that handoffs don't lose context.

Across the whole, what everyone shares is the overall goal, the direction, and the constraints, not the implementation detail of every domain. That shared picture is wider and shallower, answering different questions at different scales.

¹Matthew Skelton and Manuel Pais, *Team Topologies: Organizing Business and Technology Teams for Fast Flow* (IT Revolution, 2019).

As teams grow larger, agents multiply, and output speeds up, the gap between what the system is doing and what the team collectively understands grows, and the book doesn't pretend to have closed it. Spec Sessions narrow it. Re-alignment across domains narrows it. A platform that makes the invisible visible narrows it. None of them remove it, because the pressure toward speed and the pressure toward drift are the same force under two names.

It's worth being exact about where this model holds and where it stops. The three-role unit contains drift inside a domain: someone holds the direction, someone watches the ground, someone manages the outside, and the picture stays shared because the room is small enough to keep it shared. What it doesn't contain on its own is drift between domains: the gap that opens when ten of those rooms each stay aligned internally and slowly stop aligning with each other. No structure closes that gap by itself. It stays closed only as long as the stream-aligned teams keep choosing to re-share what they've learned, often enough, and early enough, that the divergence never compounds.

That between-domains problem is the layer where Team Topologies operates. Skelton and Pais, whose vocabulary this chapter borrows for its own domain units, map how teams should be shaped and interact at scale, so cognitive load never exceeds what a team can carry: the inter-team question of how the rooms hand off to each other. This book sits one layer down, on the understanding a single room has to build before the agent runs. Team Topologies assumes the work is already understood and optimizes the handoffs; this book argues the understanding has to be built first, and that it takes at least these three functions in the room together. The two are adjacent, not competing; cognitive load exceeding capacity and shared understanding failing to form are the same symptom seen from two layers: teams drifting, decisions made in isolation, systems nobody fully holds.

That's the honest limit. The structure buys aligned domains, not an aligned organization. The only thing that buys the second is the thing the book has been pointing at all along: a room people actually use, repeated at every level that needs one. What has to hold is the room, and the connections between rooms. Whether they exist, and whether people use them, decides everything else.